

TWO LINUX SYSTEMS FROM SCRATCH

WHAT A LINUX SYSTEM IS, HOW IT BOOTS, AND HOW DISTRO/DISTROLESS EACH BUILD ONE

AUTHOR

Javier Fernández

CONTACT

jfernand@me.com

ISSUED

2026-09-15

ABSTRACT

This workspace builds two independent, bootable Linux systems entirely from upstream source, using nothing but the host's own compiler toolchain: `distroless` (musl + BusyBox + utils, minimal) and `distro` (glibc, from-scratch, aimed eventually at a COSMIC desktop). Part 1 explains what a Linux system fundamentally **is** — the bootloader/kernel/init layering every Linux system shares, and the handful of jobs (device management, seat/session, coreutils, graphics) every desktop-capable system needs done, one way or another. Part 2 explains how this workspace's shared buildpack architecture works, and then walks through each distro's own concrete build: what packages it compiles, in what order, and how each has been verified to actually boot.

DOCUMENT CONTROL

WORKSPACE	Cargo workspace: <code>buildpack-core</code> + <code>buildpacks</code> (shared package registry) · <code>builder-tui</code> (shared dashboard) · <code>distroless</code> (musl/BusyBox) · <code>distro</code> (glibc/from-scratch) · <code>distro-init</code> (PID 1)
TARGET	Native <code>x86_64-unknown-linux-gnu</code> — host toolchain, no cross-compilation
COVERAGE	Both distros' full pipelines, kernel through a booting, logged-in shell; <code>distro</code> additionally through Weston actually rendering a client via <code>virtio-gpu</code> inside QEMU
NOT COVERED	A Rust toolchain on <code>distro</code> 's own target, COSMIC itself, real GPU drivers beyond <code>virtio-gpu</code> , audio, networking UI — see §11

PART ONE

What a Linux system is, and how it boots

01 THE THREE LAYERS

Every Linux system, regardless of distribution, is the same three layers stacked on top of each other:

LAYER	JOB
Bootloader	Runs first, finds a kernel image on disk, loads it into memory, hands off control. GRUB is the common choice; there are others.
Kernel	One binary — <code>vmlinuz</code> . Initializes hardware, mounts the root filesystem, then execs exactly one user-space program as process ID 1. Everything the kernel does before that point (driver probing, <code>devtmpfs</code> , memory management) is the same regardless of what distribution eventually boots.
Userspace	Everything else — starting with PID 1 (<code>init</code>) and everything <code>init</code> , directly or indirectly, starts after it.

What actually distinguishes one Linux system from another lives almost entirely in that third layer: which `init`, which device manager, which `coreutils` implementation, which graphics stack, if any. A minimal embedded system and a full desktop share the exact same bootloader/kernel handoff — they differ in how much userspace machinery `init` brings up afterward.

02 HOW IT BOOTS, STEP BY STEP

STEP	COMPONENT	WHAT HAPPENS
1	Bootloader	Reads the kernel image off disk and hands off.
2	Kernel	Boots, mounts the root filesystem read-write, execs <code>/sbin/init</code> .
3	<code>init</code> (PID 1)	Mounts whatever else the running system needs (<code>/proc</code> , <code>/sys</code> , <code>/dev</code> , <code>/run</code>), starts every other userspace service, and from then on supervises them — if a required service dies, PID 1 is what notices and (usually) restarts it.
4	Device management	Started by <code>init</code> , early: makes the hardware <code>devtmpfs</code> already created nodes for actually usable — permissions, symlinks, a queryable “what’s plugged in” database.
5	<code>getty</code>	Opens a terminal (the console, a serial port, both) and prints a login prompt.
6	login	Reads a username and password, checks them against the system’s account database, and — if they’re valid — replaces itself with the user’s shell.
7	shell	An interactive command interpreter. The system is now usable.

Steps 1–2 are identical on every Linux system. Steps 3 onward are where a distribution’s own choices show up — how much `init` does directly, what it delegates to separate daemons, and what those daemons are.

2.1 WHAT INIT ACTUALLY DOES, AND WHY IT’S NEVER “NOTHING”

PID 1 is not an implementation detail. The kernel will only ever execute **one** program as PID 1, and that program inherits a genuinely special job: it’s the ancestor of every other userspace process, and it’s what the kernel

hands orphaned child processes to for reaping. A system with no working init doesn't boot to a shell — it boots to a kernel panic the moment that one process exits or crashes.

Three different answers to “what should init actually do” show up across real systems:

APPROACH	WHAT IT LOOKS LIKE
systemd	The default on most desktop distributions. Starts and supervises services in dependency order, described declaratively as “unit” files, alongside a large family of companion daemons (<code>logind</code> for sessions, <code>udev</code> for devices, <code>journald</code> for logging, <code>networkd</code> , and more) that a full systemd-based system typically runs together.
A small built-in init	BusyBox ships one: reads a short, static <code>/etc/inittab</code> , starts and respawns exactly the processes it names. No dependency graph, no unit files — just a fixed list.
A purpose-written init	Nothing says init has to be either of the above. A minimal init can be a short, direct program: mount what's needed, fork a fixed set of children, wait for any of them to exit, fork it again. 99's <code>distro-init</code> is exactly this — about 100 lines.

None of these is “more correct” than the others — they're different trade-offs between flexibility (systemd can express arbitrary service dependency graphs) and simplicity (a fixed list of children is easy to read in full and reason about completely).

03

DEVICE MANAGEMENT: UDEV

The kernel's device drivers know about hardware the moment it's detected, but they don't manage `/dev` in any structured way themselves — `devtmpfs`, mounted early in boot, creates basic device nodes automatically as drivers load, and that's the extent of what it does on its own. **udev** is the userspace daemon that actually does the rest: it listens for the kernel's own hardware-change announcements (sent over a netlink socket whenever something is plugged in, unplugged, or otherwise changes state), and in response creates or removes the matching `/dev` entries, applies permissions and naming rules, and maintains a live, queryable database of what hardware currently exists.

Without something doing this job, nothing gets notified when a device appears after boot, and nothing can ask “what's connected right now” in a structured way — a real gap for anything beyond the most minimal system. `eudev` is a fork of `udev` that works without systemd (ordinary `udev` is a systemd subproject); it's one of the two device-management answers this workspace actually uses (§10, §11).

04

SEAT & SESSION MANAGEMENT

A Wayland compositor needs exclusive, arbitrated access to raw input devices (`/dev/input/*`) and the GPU (`/dev/dri/*`) — but running it as root just to get that access is the wrong trade-off. **Seat management** is the piece that solves this: a daemon holds those device files open on behalf of whichever process currently “owns” the seat, and hands out access over a socket instead. `systemd-logind` is the answer most desktop distributions use; `seatd` is a smaller, systemd-independent daemon that does the same one job and nothing else — its own documentation describes it as depending only on libc.

05

USERLAND BASICS: COREUTILS, ONE WAY OR ANOTHER

Every Unix system needs a basic set of file and text commands — `ls`, `cat`, `cp`, `mv`, `rm`, and so on. “Coreutils” is the generic name for that toolset, not one specific program, and there are several real answers to what actually provides it:

IMPLEMENTATION	WHAT IT IS
GNU coreutils	The implementation most desktop Linux distributions ship — one binary per command.
BusyBox	A much older answer aimed at tiny/embedded systems: a single small C binary that crams coreutils and a shell, <code>init</code> , <code>mount</code> , <code>ps</code> , and dozens of other tools into one multi-call executable, symlinked under every command name it implements.
utils	A from-scratch reimplementaion of just coreutils, in Rust — one multi-call binary, the same symlink-per-applet shape as BusyBox, but scoped to coreutils alone rather than a whole system toolbox.

Whichever is chosen, the shell and login program are separate concerns — coreutils gets you `ls` and `cp`, not a shell to type them into or a login prompt to authenticate at.

06

GRAPHICS, CONCEPTUALLY: DRM, MESA, GALLIUM, DRI, EGL, GBM

Getting from “a kernel that can talk to a GPU” to “a compositor rendering a client’s window” passes through several distinct layers, each solving a different part of the problem:

LAYER	WHAT IT ACTUALLY IS
DRM (kernel)	The kernel subsystem that owns the GPU at the lowest level: hands out framebuffers, submits command buffers. Has no idea what “draw a triangle” means — purely a resource-management and submission interface.
Mesa	The userspace library that turns OpenGL/OpenGL ES calls into whatever a specific GPU actually understands.
Gallium	Mesa’s own internal plumbing for doing that translation once per GPU family rather than once per API. A “Gallium driver” is the translator for one specific GPU — or, for a virtual machine, one specific virtual GPU.
DRI	Direct Rendering Infrastructure — the convention by which an application actually finds and loads the right driver at runtime.
EGL	The glue between a window system (Wayland, X11, ...) and an OpenGL/GLES context. What a compositor or client actually links against directly — not Mesa’s internals.
GBM	Generic Buffer Management — how a Wayland compositor allocates the actual pixel buffers it hands to the DRM/KMS display hardware. The piece that makes a rendered frame show up on screen rather than just existing in GPU memory.

A minimal Wayland-only graphics stack built from source needs, at minimum: the kernel’s DRM driver for the target GPU, `libdrm` (the kernel-userspace ioctl wrapper every one of these libraries builds on), and Mesa itself providing `libEGL` / `libGL` / `libGLESv2` / `libgbm` plus at least one Gallium driver. `llvmpipe` (an LLVM-based software rasterizer) is one possible fallback driver, not a hard requirement — `virgl` (talks to a virtual machine’s `virtio-gpu/virgl` backend) and `softpipe` (a non-LLVM software fallback) are two others that work without pulling LLVM into the build at all.

07

STATIC VS DYNAMIC LINKING

A binary can be built two ways: **statically linked**, where every library it needs is compiled directly into the one executable file, or **dynamically linked**, where the executable instead records which shared libraries (`.so` files) it needs and expects to find them on the target system at run time. Static linking means “compile once, copy one file, done” — no shared-library bookkeeping on the target at all — but it only scales to programs whose dependencies are genuinely small and self-contained. Anything that needs a library with real runtime state or plugin-style extensibility (D-Bus, a graphics driver stack) is effectively required to link

dynamically, which means the target system needs those shared libraries physically present and findable by the dynamic linker before the program will run at all.

PART TWO

How distroless and distro each build one

08 THIS WORKSPACE

CRATE	ROLE
buildpack-core	Foundational library: the <code>Buildpack</code> trait every package implements, <code>DistroConfig</code> (the shared config-file format), the <code>PipelineStage</code> trait for whole-image operations, and the dependency-graph/build helpers both distros use.
buildpacks	One implementation per upstream package — kernel, coreutils, every library in §10–11's tables — shared between whichever distro(s) actually use each one.
builder-tui	A generic interactive terminal dashboard for running any stage of either distro's pipeline and writing the result to a USB stick.
distroless	musl + BusyBox + utils — a small, minimal distro. §10.
distro	A from-scratch glibc system, native-compiled, aimed eventually at a COSMIC desktop. §11.
distro-init	A from-scratch PID 1 written for this project, ~100 lines of Rust — used by <code>distro</code> (§11.1).

8.1 THE SHARED BUILDPACK ARCHITECTURE

Every upstream package this workspace builds — for either distro — is one self-contained value implementing the `Buildpack` trait: it declares its own source location, its dependencies on other buildpacks, how to fetch and build it, and what files it produces.

CONCEPT	WHAT IT DOES
Buildpack trait	<code>fetch()</code> / <code>build()</code> / <code>outputs()</code> / <code>clean()</code> plus a <code>describe()</code> for the TUI. One implementation per upstream package, in <code>buildpacks</code> .
DistroConfig	The shared config-file format both <code>distro.toml</code> and <code>distroless.toml</code> load through — <code>build_dir</code> , <code>networking</code> , and <code>image</code> are the only fields it understands directly; every other section is handed, unparsed, straight to that package's own buildpack.
Dependency graph	A real topological sort over every buildpack's declared dependencies decides build order — not a hand-maintained call sequence. Rendered fresh to an SVG (<code>dependency-graph.svg</code>) on every build.
PipelineStage trait	For the three operations that act on the whole assembled image rather than any one package: partitioning and writing the disk image, booting it in QEMU, and writing it to a USB stick. Identical logic for both distros.
CLI subcommands	<code>list-packages</code> , <code>fetch-pkg <id></code> , <code>build-pkg <id></code> , <code>clean-pkg <id></code> — inspect or rebuild one package at a time, on either distro.
builder-tui	An interactive dashboard driven by the same registry of buildpacks — shows build status per package, runs any stage, walks through settings (networking, kernel features, hostname), and writes to a USB stick with a confirmation prompt.

Each package's config lives under a `[section]` named after its buildpack id — a version and a source URL, at minimum. `distro.toml` and `distroless.toml` differ only in which sections they define, not in format.

09

HOW DISTROLESS BUILDS A LINUX SYSTEM

`distroless` is deliberately the smallest of the two: `musl libc`, `BusyBox` for the entire `coreutils/shell/init/` device-management job it can cover, plus `utils` and the kernel as the two things built from source beyond `BusyBox` itself. Everything in it is cross-compiled against `musl` rather than the host's own `glibc`.

9.1 BOOT SEQUENCE

STEP	COMPONENT	WHAT HAPPENS
1-2	GRUB, kernel	Same as §2's generic sequence.
3	BusyBox init	Reads <code>/etc/inittab</code> : runs <code>/etc/init.d/rcS</code> once at boot, then starts and respawns <code>getty</code> on the console.
4	<code>rcS</code>	A short shell script: mounts <code>/proc / /sys / devtmpfs</code> , sets the hostname, and — if networking is enabled — brings up <code>lo</code> and runs <code>udhcpc</code> for a DHCP lease.
5-7	<code>getty</code> → <code>login</code> → <code>shell</code>	BusyBox's own <code>getty / login / ash</code> — a single root account with no password set.

9.2 PACKAGES

PACKAGE	PROVIDES	NOTES
Linux kernel	<code>vmlinuz</code>	The same <code>Kernel</code> buildpack <code>distro</code> uses — kernel builds have no <code>musl/glibc</code> -specific behavior.
<code>utils/coreutils</code>	<code>ls</code> , <code>cat</code> , <code>cp</code> , ...	The <code>musl</code> variant of the same <code>Utils</code> buildpack <code>distro</code> uses (§11.4) — one multi-call binary, statically linked, <code>musl-static</code> by default.
BusyBox	<code>shell</code> , <code>init</code> , <code>mount</code> , <code>getty</code> , and the rest of a minimal system toolbox	Built via a curated <code>Kconfig</code> : <code>allnoconfig</code> , then a specific applet list enabled and cross-compiled against <code>musl</code> , statically linked.

9.3 PIPELINE

DISTROLESS'S CLI SURFACE

```

distroless fetch           # download + extract kernel, utils, busybox
distroless build-toolchain # install musl-gcc + the musl Rust target (once)
distroless build-kernel   # the kernel buildpack
distroless build-userland  # utils + busybox
distroless assemble-rootfs # merge it all into build/rootfs
distroless make-image      # partition + GRUB + write the disk image
distroless test-qemu [--window] # boot it
distroless write-usb --device <dev> # dd to real hardware (destructive)
distroless tui             # interactive dashboard for all of the above
distroless all             # the whole pipeline, in order

```

Assembling the `rootfs` copies `utils`' and `BusyBox`'s declared outputs in directly (`utils`' applet symlinks, `BusyBox`'s own `bin/sh`, `sbin/init`, and applet `set`), installs the kernel modules this build produced, and writes the handful of config files `BusyBox`'s `init` expects (`/etc/inittab`, `/etc/passwd`, `/etc/fstab`, `/etc/init.d/rcS`, and — when networking is enabled — a `udhcpc` `bound/renew/deconfig` script).

VERIFIED
A scripted QEMU boot reaches the login prompt, logs in via BusyBox `ash` with no password, and confirms `ls /bin | wc -l` and a correct `uname -a` against the just-built kernel.

10

HOW DISTRO BUILDS A LINUX SYSTEM

`distro` is the larger of the two: a from-scratch **glibc** system, native-compiled against the host’s own tool-chain — no cross-compilation — aimed eventually at running a COSMIC desktop. Where `distroless` reaches for one BusyBox binary to cover `coreutils`, `shell`, `init`, and device management all at once, `distro` builds a separate, purpose-specific package for each of those jobs, plus (currently) the full graphics stack needed to host a Wayland compositor.

10.1 BOOT SEQUENCE

STEP	COMPONENT	WHAT HAPPENS
1-2	GRUB, kernel	Same as §2’s generic sequence.
3	distro-init	Our own PID 1 (below). Mounts <code>/proc</code> , <code>/sys</code> , <code>/dev</code> (<code>devtmpfs</code>), and a fresh <code>tmpfs</code> at <code>/run</code> .
4	udev, systemd, dbus-daemon	Forked and exec’d, in that order. <code>distro-init</code> then runs <code>udevadm trigger</code> once — a coldplug that tells <code>udev</code> about the devices <code>devtmpfs</code> already created before it started.
5	agetty ×2	Forked on <code>ttys0</code> (VGA console) and <code>ttys0</code> (serial, for QEMU/headless use). Whichever exits gets respawned.
6	login	<code>agetty</code> execs <code>/bin/login</code> once a username is typed. Reads <code>/etc/passwd</code> / <code>/etc/shadow</code> for real — the <code>root</code> account’s shadow entry has an empty password field, so <code>login</code> skips the password prompt.
7	bash	<code>login</code> execs the shell named in <code>/etc/passwd</code> .

VERIFIED LIVE
Every arrow above was reproduced in QEMU with a scripted serial-console session: `login: root` → no password prompt → `-bash-5.2#` → `dbus-send --system ... ListNames` returns a real reply → `udevadm info` returns a populated device entry.
`--query=all --name=/dev/tty1`

distro-init
Nothing **glibc**-based has a BusyBox-equivalent single `init` binary, and pulling in `systemd` would drag in `logind`, `udev`, `journald`, and far more than this system currently needs. `distro-init` is a purpose-built `init` instead — about 100 lines of Rust using the `nix` crate — following exactly the “purpose-written `init`” pattern described in §3.1.

DISTRO-INIT/SRC/MAIN.RS – THE WHOLE SUPERVISION LOOP, ABBREVIATED

```
fn main() {
    mount_basic_filesystems();           // proc, sysfs, devtmpfs, tmpfs at /run

    let mut udevd_pid = spawn_udev();    // no -d: stays a tracked child, not a daemon
    let mut seatd_pid = spawn_seatd();
    let mut dbus_pid = spawn_dbus();     // --nofork, same reasoning
    coldplug_devices();                  // `udevadm trigger`, one-shot

    let mut ttyl_pid = spawn_ttyl();     // agetty
    let mut serial_pid = spawn_serial();

    loop {
        match waitpid(None, Some(WaitPidFlag::empty())) {
            Ok(WaitStatus::Exited(pid, _)) | Ok(WaitStatus::Signaled(pid, _, _)) => {
                // whichever of the five died gets forked again; anything
                // else reaped here was just an orphaned grandchild.
            }
            ...
        }
    }
}
```

Every daemon it starts would normally daemonize itself (fork, detach, exit the parent) — `dbus-daemon --nofork` and `udev` with `no -d` suppress that, so they stay `distro-init`'s direct children and its `waitpid` loop actually sees them exit if they crash.

10.2 THE KERNEL

`distro.toml` names a kernel version and a download URL under `[kernel]`; `distro build-kernel` fetches, configures, and compiles it. The starting config is either `make defconfig` (the default) with every optional **feature pack** below stripped back off, or a previously saved `.config` from an interactive `make menuconfig` session (`distro menu-config`), re-resolved via `make olddefconfig`. Either way, whatever packs are named in `kernel.features` get turned back on as the final step.

PACK	WHAT IT TURNS ON
graphics	DRM/KMS graphics + fbdev console (i915, virtio-gpu, bochs, AGP) instead of plain VGA text.
sound	ALSA sound subsystem and the Intel HDA driver.
wireless	The Wi-Fi stack (<code>cfg80211/mac80211</code>) and <code>rkill</code> .
hid-extras	Per-vendor HID quirk drivers and the <code>hiddev/hidraw</code> userspace interfaces — generic USB HID keyboards/mice work without this.
legacy-nics	Dedicated Ethernet chipset drivers for real hardware — QEMU's <code>virtio-net</code> always works without this.
legacy-buses	PCMCIA/CardBus and legacy PATA chipset drivers.
network-fs	NFS (client + root-over-NFS), 9P, and autofs.
netfilter	Connection tracking, NAT, and iptables.
security-extras	Disk quotas, POSIX ACLs, and SELinux.
iommu	AMD/Intel IOMMU support — PCI passthrough or virtualization-host use.
debug	Kernel debug instrumentation — useful while bringing up boot, dead weight once stable.
ia32-emulation	Run 32-bit x86 binaries on this 64-bit kernel.
iso9660	ISO9660/Joliet/zisofs filesystem support.
boot-logo	Framebuffer console + boot-time logo (stock penguin, or a custom 80×80 PPM via <code>kernel.logo_file</code>).

`distro.toml` currently sets no `kernel.features` at all — the kernel ships the fully stripped `defconfig` baseline plus whatever `graphics` support the rootfs actually needs is enabled directly, not via a named pack.

10.3 PACKAGES

Five packages exist purely to get from a mounted rootfs to a real, authenticated shell — the `glibc` equivalent of what `BusyBox` does as one binary, spread across separate upstream projects because `glibc`-land has no single equivalent to reach for. All five are statically linked: one binary compiled, one binary copied into the rootfs, nothing else to track.

PACKAGE	VERSION	PROVIDES	WHY IT'S THERE
<code>utils/coreutils</code>	<code>git main</code>	<code>ls</code> , <code>cat</code> , <code>cp</code> , ...	Rust reimplementation of GNU <code>coreutils</code> , chosen over <code>BusyBox</code> for this distro specifically to prefer Rust alternatives where practical.
<code>bash</code>	5.2.37	<code>/bin/bash</code> , <code>/bin/sh</code>	The login shell named in <code>/etc/passwd</code> .
<code>util-linux</code>	2.41.2	<code>agetty</code> , <code>mount</code> , <code>umount</code>	Built with only these three programs enabled — <code>util-linux</code> ships dozens, only these are needed here.
<code>shadow-utils</code>	4.17.4	<code>login</code> , <code>passwd</code>	Real <code>/etc/passwd</code> + <code>/etc/shadow</code> authentication.

Three daemons do what a desktop session needs before any compositor can run — none of them optional, the floor a compositor stands on rather than decoration:

PACKAGE	VERSION	WHAT IT ACTUALLY DOES
<code>seatd</code>	0.9.3	Owns <code>/dev/input/*</code> and <code>/dev/dri/*</code> on behalf of whatever process asks over its socket — the seat-management daemon described conceptually in §4, chosen over <code>systemd-logind</code> to avoid pulling in the rest of <code>systemd</code> .
<code>dbus</code>	1.16.2	The system message bus. Runs as <code>root</code> here — the rootfs has no unprivileged <code>messagebus</code> user yet to drop privileges to.
<code>eudev</code>	3.2.14	The <code>systemd</code> -independent <code>udev</code> fork described in §3. Exists in this pipeline specifically because <code>libinput</code> hard-depends on <code>libudev</code> .

Seven libraries sit ready for a compositor to link against — nothing in this group runs on its own; verification at this layer is “builds and installs cleanly,” not “does something observable”:

PACKAGE	VERSION	WHAT IT'S FOR
<code>wayland</code>	1.26.0	The core wire-protocol libraries and <code>wayland-scanner</code> , the code generator every later Wayland package needs at its own build time.
<code>wayland-protocols</code>	1.49	The actual protocol definitions — <code>xdg-shell</code> and the rest — as XML.
<code>libxkbcommon</code>	1.12.4	Turns “us, evdev, pc105” into the keymap tables a compositor hands to clients.
<code>pixman</code>	0.46.4	Software rasterization — Mesa's fallback path.
<code>libdisplay-info</code>	0.4.0	Parses a monitor's own EDID/DisplayID.
<code>libevdev</code>	1.13.7	Reads and writes raw <code>evdev</code> input-device events — <code>libinput</code> 's one mandatory dependency.
<code>libinput</code>	1.31.3	Turns raw <code>evdev</code> events into pointer/keyboard/touch/gesture events a compositor actually wants.

The graphics stack proper — from §6's conceptual layering down to actual built libraries, scoped to QEMU's own virtual GPU rather than real hardware:

PACKAGE	VERSION	WHAT IT'S FOR
<code>libdrm</code>	2.4.134	The kernel-userspace <code>ioctl</code> wrapper every GPU-facing library builds on — every vendor-specific sub-library disabled, <code>virtio-gpu</code> needs only the generic core.

PACKAGE	VERSION	WHAT IT'S FOR
Mesa	26.2.2	libEGL, libGLv2, libgbm, and the Gallium driver — built scoped to virgl (talks to QEMU's virtio-gpu/virgl backend) and softpipe (software fallback). Vulkan and GLX/X11 both disabled — Wayland/EGL/GLES only, no LLVM needed for either driver.

Hosting an actual compositor needs six more packages, plus a keyboard layout data package:

PACKAGE	VERSION	WHAT IT'S FOR
zlib	1.3.2	Compression — libpng's and cairo's own dependency.
expat	2.7.1	XML parsing — fontconfig's config-file dependency.
libpng	1.6.44	PNG images — Weston's own unconditional dependency, and cairo's PNG surface backend.
FreeType	2.13.3	Font rasterization.
fontconfig	2.15.0	Font matching — FreeType's companion, needed for cairo's font backend.
cairo	1.18.2	2D graphics — Weston's mandatory window-decoration dependency. Image-surface only, no X11/xcb/GL backends.
xkeyboard-config	2.44	Keyboard layout/rules data — no code, what libxkbcommon looks for at runtime.
Weston	16.0.0	The compositor itself — DRM backend, GL renderer, kiosk shell, weston-simple-egl. Vulkan/X11/Xwayland/systemd/JPEG/WebP/LCMS2 all disabled.

VERIFIED — THE ACTUAL MILESTONE

A real QEMU boot: Weston (launched with a minimal `weston.ini` selecting its kiosk shell) initializes DRM, EGL/GL via this workspace's own Mesa softpipe Gallium driver, libinput, and the kiosk shell, and opens a Wayland socket. `weston-simple-egl`, run against that socket, connects and renders without error — a minimal Wayland client actually rendering via virtio-gpu inside QEMU. Not yet automatic at boot: today Weston is started by hand from a login shell, not a `distro-init` service.

10.4 THE SYSROOT: HOW THESE PACKAGES FIND EACH OTHER

Static-base packages (§11.4's first table) never need each other at build time — each just needs the host's gcc. Everything from the seat layer onward does: `wayland-protocols` needs `wayland-scanner` on PATH at its own build time, and `libinput` needs `eudev`'s installed `libudev.pc` to link against `libudev` at all. Every one of those packages is therefore built with `--prefix=/usr` (its normal, final, “as if genuinely installed” prefix) and installed with `DESTDIR=<sysroot>` — files physically land under a shared sysroot directory, but each package's own compiled-in idea of its prefix stays `/usr`, which matters: some of these daemons look up their own config files relative to whatever prefix they were **actually built with**, at their own runtime, on the real target. `PKG_CONFIG_SYSROOT_DIR` (pkg-config's own mechanism for exactly this case) rewrites the `-I / -L` paths a later package's build sees from `/usr/...` to the sysroot's real, on-disk `<sysroot>/usr/...`. Assembling the rootfs then copies that whole sysroot tree in with one `cp -a`.

10.5 PIPELINE

DISTRO'S CLI SURFACE

```
distro fetch           # download + extract every source
distro build-toolchain # apt-get the host build tools (once)
distro build-kernel    # the kernel buildpack
distro build-userland   # every other package
distro assemble-rootfs  # merge it all into build-distro/rootfs
distro make-image       # partition + GRUB + write the disk image
distro test-qemu [--window] # boot it
distro write-usb --device <dev> # dd to real hardware (destructive)
distro tui              # interactive dashboard for all of the above
distro all              # the whole pipeline, in order
```

`build-userland` fetches and builds every non-kernel package in dependency order (§9.1's topological sort); `assemble-rootfs` then copies every statically-linked package's declared outputs in directly (`coreutils` and its applet symlinks, `bash + bin/sh`, `util-linux's agetty / mount / umount`, `shadow's login / passwd`, `distro-init` as `/sbin/init`), bulk-copies the whole `sysroot` for every dynamically-linked package in one `cp -a`, then adds the host's own C library, `libstdc++`, and the handful of other runtime `.so`s these dynamically-linked packages need, plus the dynamic linker itself and `/etc/passwd + /etc/shadow`.

VERIFIED

A full pipeline run (`fetch → build-kernel → build-userland → assemble-rootfs → make-image`) reports every package “already exists” on a rebuild, and a full QEMU regression boot — `login`, `dbus-send`, `udevadm`, and Weston's own DRM/EGL/GL initialization — passes end to end.

11

WHAT ISN'T PART OF THE PICTURE YET

NEXT `rustup / cargo` on-target, plus a curated Rust-CLI-tools suite (`ripgrep`, `bat`, `eza`, ...) — no new architecture needed, each is just another buildpack.

THEN COSMIC itself — `cosmic-comp`, `cosmic-session`, `cosmic-panel`, `cosmic-greeter`, minimal subset first.

LATER More COSMIC components, real GPU drivers beyond `virtio-gpu`, audio, networking UI.

And three deliberate gaps in what's already built, worth knowing about rather than discovering later:

- `utils`' built feature set does not include `grep` or `sed` — they were never part of `coreutils`' scope upstream in the first place. The applet symlinks exist but dangle. Not yet fixed.
- `libxcb` was never built. Its only real consumer on a Wayland-native target is XWayland compatibility, not currently planned.
- `mtdev` (legacy multitouch) and `libwacom` (tablet identification) were both left out of `libinput` — niche hardware support, easy to add back later if a real device needs it.